Constraint Satisfaction Problems Study Notes

Part 3: Backtracking Search

Learning Goal

By the end of this part, you should understand how backtracking search solves CSPs, why it is better than a naive search formulation, and how partial assignments are expanded step by step until a complete consistent solution is found or failure is detected.

1. Core Concept

Backtracking search is the basic systematic method for solving CSPs. It builds a solution incrementally by assigning values to variables one at a time. If a partial assignment leads to a contradiction, the algorithm undoes the most recent assignment and tries another option.

This means the algorithm does not keep exploring a bad path once it becomes inconsistent. It goes back, changes the decision, and continues the search.

> **Things to Note:**
>
> - Backtracking is the baseline uninformed algorithm for CSPs.
> - It is a depth-first search method specialized for variable assignment problems.
> - It works with **partial assignments**, not complete random states.

2. The Straightforward Search Formulation

Suppose we try to solve a CSP using a very direct search view.

- The initial state is the empty assignment.
- A successor is produced by assigning a value to an unassigned variable.
- The goal is any complete assignment satisfying all constraints.

This sounds reasonable, but if treated too generally, it creates a massive search tree.

At depth l, there are still many unassigned variables and many domain values, so the branching factor can become very large. If we do not use the structure of CSPs properly, search wastes effort exploring different orders of assignments that are not meaningfully different.

Why naive search is wasteful

Imagine assigning the same set of variable values in different orders. In most CSPs, these represent the same partial solution logically, but a naive search process may still treat them as different branches.

This creates unnecessary duplication.

> **Main Problem with Naive Search:**
>
> If assignment order is treated as important when it really is not, the search tree becomes much larger than necessary.

3. The Key Insight: Assignments Are Commutative

In CSPs, variable assignments are usually **commutative**. This means that assigning:

- `WA = red` and then `NT = green`

is logically the same as assigning:

- `NT = green` and then `WA = red`

The final partial assignment is identical. The order used to reach it does not matter.

This is the reason backtracking is more efficient than naive search. It avoids exploring all meaningless permutations of the same assignments.

> **High-Yield Point:**
>
> The most important reason backtracking improves over naive search is that it exploits the fact that assignment order is commutative.

4. How Backtracking Search Works

Backtracking follows a simple pattern:

1. Start with an empty assignment.
2. Choose an unassigned variable.
3. Try one of its possible values.
4. If the value is consistent with the assignment so far, keep it and move forward.
5. If no value works, backtrack to the previous variable and try a different value there.
6. Continue until a complete consistent assignment is found or all possibilities fail.

Core idea in one sentence

Backtracking is depth-first search with one variable assigned per step and inconsistency used as the signal to reverse course.

5. Standard Backtracking Procedure

The logic of the standard procedure can be described like this:

1. If the assignment is complete, return it as the solution.
2. Select an unassigned variable.

3. Try the domain values for that variable one by one.
4. If a value does not violate any constraint with the current assignment, add it.
5. Recursively continue from the new partial assignment.
6. If recursion fails, remove that value and try the next one.
7. If all values fail, return failure to the previous level.

```
BACKTRACK(assignment):
    if assignment is complete:
        return assignment
    choose an unassigned variable X
    for each value v in DOMAIN(X):
        if X = v is consistent with assignment:
            add X = v to assignment
            result = BACKTRACK(assignment)
            if result is not failure:
                return result
            remove X = v from assignment
    return failure
```

This pseudocode is short, but it captures the whole idea of systematic CSP search.

> **Common Confusion:**
>
> Students often think backtracking means “start over from scratch.” It does not. It means undoing the most recent decision that still has unexplored alternatives and trying a new option from there.

6. Example A: Backtracking in Map Coloring

Consider a map-coloring problem with variables representing regions and domain values `{red, green, blue}`.

Suppose the algorithm proceeds like this:

1. Assign `WA = red`
2. Assign `NT = green`
3. Try `SA = red`

At this point, if `SA = red` conflicts with neighboring assignments, then that value is rejected. The algorithm will then try another color for `SA`, such as `blue`.

If all colors for `SA` fail, then the algorithm backtracks to the previous variable, for example `NT`, and tries another value there.

What this teaches

- The algorithm moves forward only when consistency is preserved.

- Failure at one point does not end the whole search.
- It simply triggers reconsideration of earlier choices.

7. Example B: Backtracking in Exam Scheduling

Now use the exam scheduling example with:

- `CS101`
- `CS102`
- `MATH201`
- `ENG101`

and domain:

`{Mon-AM, Mon-PM, Tue-AM}`

with hard constraints:

- `CS101 ≠ CS102`
- `CS101 ≠ MATH201`
- `CS102 ≠ ENG101`
- `ENG101 ≠ Mon-AM`

A possible backtracking trace

1. Assign `CS101 = Mon-AM`
2. Try `CS102 = Mon-AM` → reject because `CS101 ≠ CS102`
3. Try `CS102 = Mon-PM` → accept
4. Try `MATH201 = Mon-AM` → reject because `CS101 ≠ MATH201`
5. Try `MATH201 = Tue-AM` → accept
6. Try `ENG101 = Mon-AM` → reject because `ENG101 ≠ Mon-AM`
7. Try `ENG101 = Mon-PM` → reject because `CS102 ≠ ENG101`
8. Try `ENG101 = Tue-AM` → accept

The result is a complete valid assignment:

- `CS101 = Mon-AM`
- `CS102 = Mon-PM`
- `MATH201 = Tue-AM`
- `ENG101 = Tue-AM`

This assignment is valid because all hard constraints are satisfied.

Why this example matters

It shows clearly how consistency checking prevents illegal choices from being extended further.

8. Backtracking Uses Partial Assignments

At any intermediate point, backtracking maintains a **partial assignment**. This means:

- some variables are assigned,
- some are still unassigned,
- the current assignment is consistent so far.

This is important because CSP solving does not usually start from a complete state. It grows a solution piece by piece.

Type of Assignment	Meaning
Partial assignment	Only some variables are assigned
Consistent partial assignment	No constraints are violated so far
Complete assignment	All variables are assigned
Solution	A complete assignment that satisfies all hard constraints

> **Common Confusion:**
>
> A partial assignment can be consistent without being a solution. Students often stop mentally too early. A branch is only successful when the assignment is both complete and consistent.

9. Why Backtracking Is Better Than Naive Search

Backtracking improves efficiency in two major ways:

- It assigns only one variable at a time.
- It avoids treating different assignment orders as different meaningful states.

This greatly reduces the number of branches that need to be explored.

It is still exponential in the worst case, but it is far better than the fully naive approach.

Things to Note:

- Backtracking is not magically fast. Worst-case complexity is still exponential.
- Its importance is that it gives a clean systematic framework that can later be improved with heuristics and propagation.
- Think of backtracking as the foundation on which smarter CSP methods are built.

10. When Does Backtracking Fail?

Backtracking fails along a branch when the algorithm reaches a variable for which no value can be assigned consistently with the current partial assignment.

When that happens:

- the current branch is impossible,
- the algorithm returns to an earlier variable,
- and it tries another value there.

If every possibility is exhausted, the entire CSP has no solution.

Example idea

If in exam scheduling every remaining slot for `ENG101` is blocked by either instructor unavailability or exam conflict, then the current branch fails and must be abandoned.

11. What Backtracking Does Not Yet Do

Basic backtracking is systematic, but still simple. At this stage it does not yet answer questions like:

- Which variable should be assigned first?
- Which value should be tried first?
- Can inevitable failure be detected earlier?

Those improvements come later through heuristics and constraint propagation.

So this part gives the basic engine. Later parts make that engine smarter.

Bridge to the Next Part:

Backtracking gives correctness and structure. Heuristics will later improve efficiency by choosing smarter variables and values.

12. Things to Think About

- Why is depth-first search a natural fit for backtracking in CSPs?
- Why does the commutativity of assignments reduce wasted search?

- Why is rejecting inconsistent values early better than completing the whole assignment first?
- What kind of problem would make plain backtracking very slow?

13. Quick Recap

- Backtracking is the basic depth-first search method for CSPs.
- It builds solutions incrementally through partial assignments.
- Assignments are commutative, so order does not need to be treated as part of the solution path.
- When a choice leads to inconsistency, the algorithm backs up and tries another option.
- A consistent partial assignment is not yet a solution.
- Basic backtracking is systematic but still limited until heuristics and propagation are added.

14. Part 3 Revision Lines

> - Backtracking = **DFS + one-variable-at-a-time assignment**.
> - Main advantage over naive search = **assignment order is commutative**.
> - Main weakness = **still exponential without smarter guidance**.

End of Part 3